

Software Engineering (U14PC601)

Introduction to Software Engineering (SE)

Software Engineering is a systematic, disciplined, and measurable approach to the **development, operation, maintenance, and evolution** of software.

Definition:-

Software Engineering is the application of **engineering principles** to software development to produce **high-quality, reliable, efficient, and maintainable software** within time and cost constraints.

Key Goals

- Deliver **correct and reliable software**
- Meet **user requirements**
- Reduce **development cost and time**
- Improve **quality and maintainability**

Importance of Software Engineering

Software Engineering is important because modern software systems are large, complex, and critical.

Importance / Need:-

- **Handles Complexity**
 - Large systems are divided into manageable modules.
- **Improves Software Quality**
 - Ensures reliability, efficiency, security, and performance.
- **Reduces Cost and Time**
 - Proper planning and design avoid rework and failures.
- **Ensures Maintainability**
 - Software can be easily updated and fixed.
- **Supports Team Development**
 - Provides standards, documentation, and processes.
- **Manages Risks**
 - Identifies problems early in development.

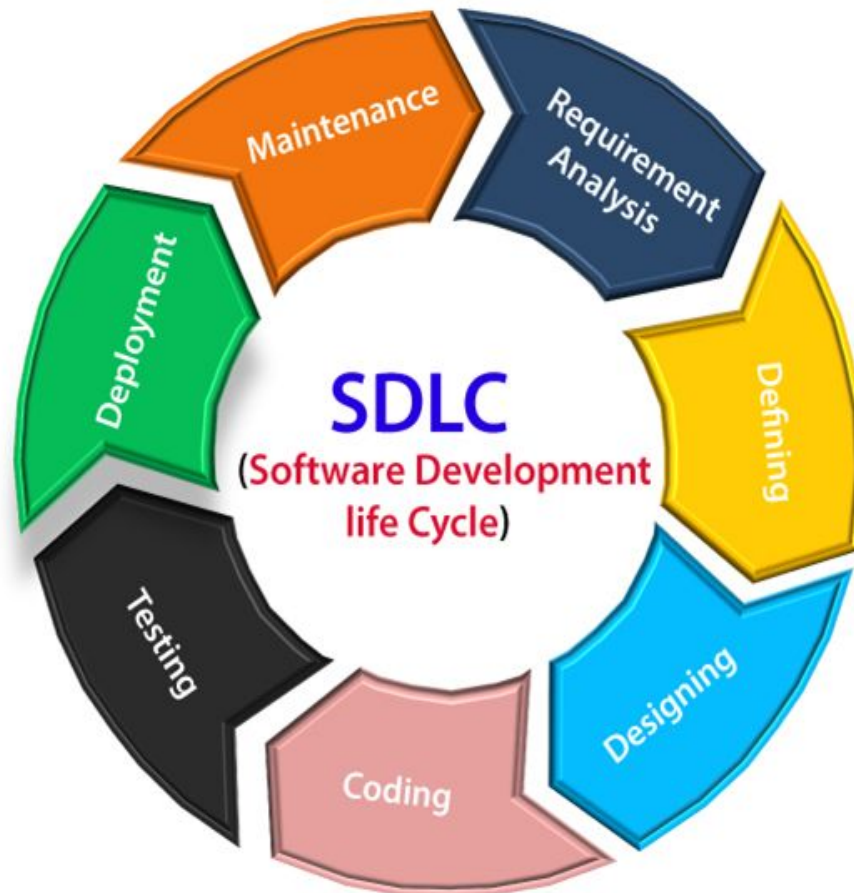
SDLC (Software Development Life Cycle)

Purpose of SDLC

- Ensure systematic development
- Improve product quality
- Control project progress
- Meet user requirements

SDLC

- Software Development Life Cycle (SDLC)
- SDLC Cycle represents the process of developing software. SDLC framework includes the following steps:



Stage1: Planning and requirement analysis

Requirement Analysis is the most important and necessary stage in SDLC.

The senior members of the team perform it with inputs from all the stakeholders and domain experts or SMEs in the industry.

Planning for the quality assurance requirements and identifications of the risks associated with the projects is also done at this stage.

Business analyst and Project organizer set up a meeting with the client to gather all the data like what the customer wants to build, who will be the end user, what is the objective of the product. Before creating a product, a core understanding or knowledge of the product is very necessary.

For Example, A client wants to have an application which concerns money transactions. In this method, the requirement has to be precise like what kind of operations will be done, how it will be done, in which currency it will be done, etc.

- Once the required function is done, an analysis is complete with auditing the feasibility of the growth of a product. In case of any ambiguity, a signal is set up for further discussion.
- Once the requirement is understood, the SRS (Software Requirement Specification) document is created. The developers should thoroughly follow this document and also should be reviewed by the customer for future reference.

Stage2: Defining Requirements

Once the requirement analysis is done, the next stage is to certainly represent and document the software requirements and get them accepted from the project stakeholders.

This is accomplished through "SRS"- Software Requirement Specification document which contains all the product requirements to be constructed and developed during the project life cycle.

Stage3: Designing the Software

The next phase is about to bring down all the knowledge of requirements, analysis, and design of the software project. This phase is the product of the last two, like inputs from the customer and requirement gathering.

Stage4: Developing the project

In this phase of SDLC, the actual development begins, and the programming is built. The implementation of design begins concerning writing code. Developers have to follow the coding guidelines described by their management and programming tools like compilers, interpreters, debuggers, etc. are used to develop and implement the code.

Stage5: Testing

After the code is generated, it is tested against the requirements to make sure that the products are solving the needs addressed and gathered during the requirements stage.

During this stage, unit testing, integration testing, system testing, acceptance testing are done.

Stage6: Deployment

Once the software is certified, and no bugs or errors are stated, then it is deployed.

Then based on the assessment, the software may be released as it is or with suggested enhancement in the object segment.

After the software is deployed, then its maintenance begins.

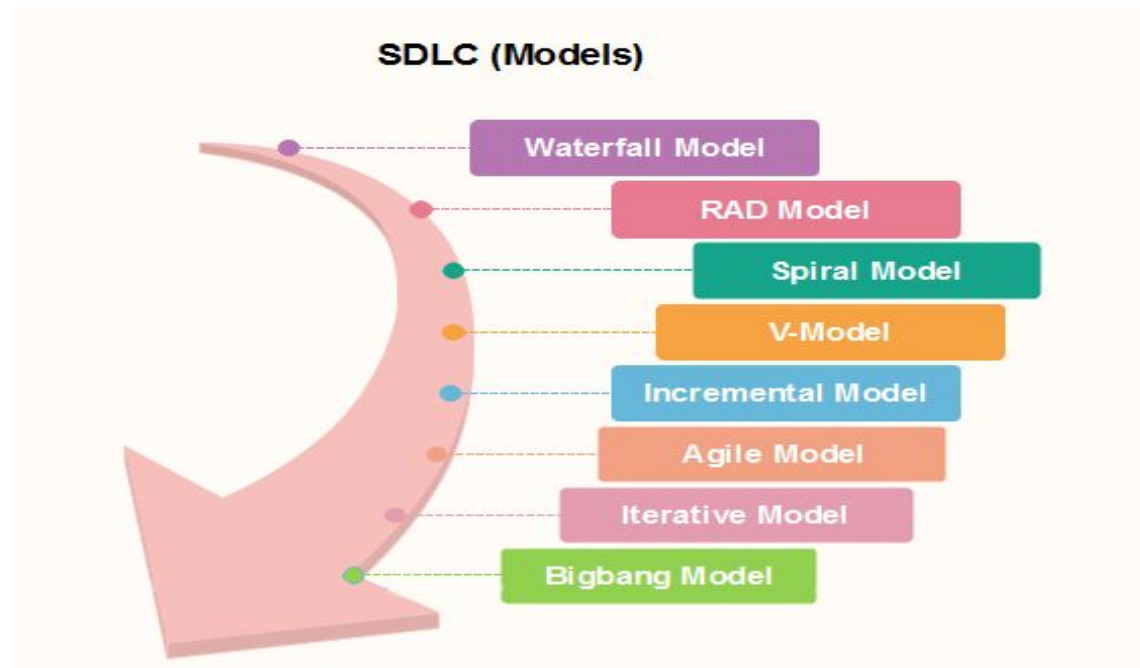
Stage7: Maintenance

Once when the client starts using the developed systems, then the real issues come up and requirements to be solved from time to time.

This procedure where the care is taken for the developed product is known as maintenance.

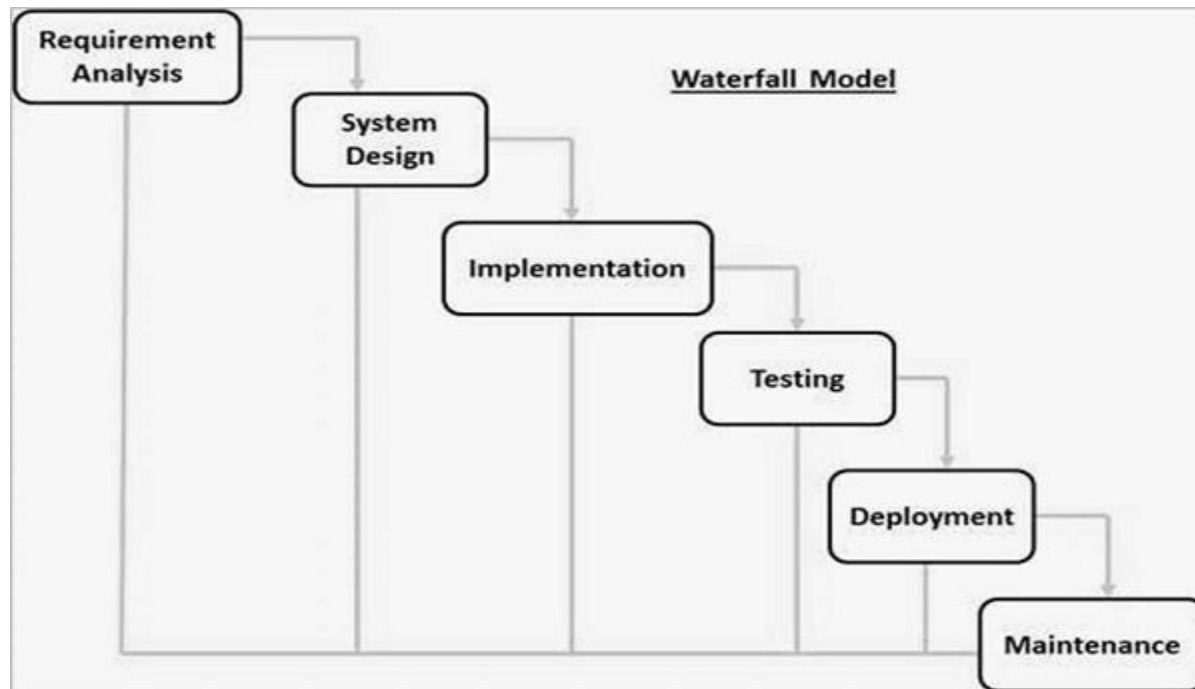
SDLC Models

- Software Development life cycle (SDLC) is a spiritual model used in project management that defines the stages include in an information system development project, from an initial feasibility study to the maintenance of the completed application.
- There are different software development life cycle models specify and design, which are followed during the software development phase. These models are also called "**Software Development Process Models.**" Each process model follows a series of phase unique to its type to ensure success in the step of software development.



Waterfall model

- Winston Royce introduced the Waterfall Model in 1970. This model has five phases: Requirements analysis and specification, design, implementation, and unit testing, integration and system testing, and operation and maintenance. The steps always follow in this order and do not overlap. The developer must complete every phase before the next phase begins. This model is named "**Waterfall Model**", because its diagrammatic representation resembles a cascade of waterfalls.



- **Requirement Gathering and analysis** – All possible requirements of the system to be developed are captured in this phase and documented in a requirement specification document.
- **System Design** – The requirement specifications from first phase are studied in this phase and the system design is prepared. This system design helps in specifying hardware and system requirements and helps in defining the overall system architecture.
- **Implementation** – With inputs from the system design, the system is first developed in small programs called units, which are integrated in the next phase. Each unit is developed and tested for its functionality, which is referred to as Unit Testing.
- **Integration and Testing** – All the units developed in the implementation phase are integrated into a system after testing of each unit. Post integration the entire system is tested for any faults and failures.
- **Deployment of system** – Once the functional and non-functional testing is done; the product is deployed in the customer environment or released into the market.
- **Maintenance** – There are some issues which come up in the client environment. To fix those issues, patches are released. Also to enhance the product some better versions are released. Maintenance is done to deliver these changes in the customer environment.
- All these phases are cascaded to each other in which progress is seen as flowing steadily downwards (like a waterfall) through the phases. The next phase is started only after the defined set of goals are achieved for previous phase and it is signed off, so the name "Waterfall Model". In this model, phases do not overlap.

Some of the major advantages of the Waterfall Model are as follows –

- Simple and easy to understand and use
- Easy to manage due to the rigidity of the model. Each phase has specific deliverables and a review process.
- Phases are processed and completed one at a time.
- Works well for smaller projects where requirements are very well understood.
- Clearly defined stages.
- Well understood milestones.
- Easy to arrange tasks.
- Process and results are well documented.

The major disadvantages of the Waterfall Model are as follows –

- No working software is produced until late during the life cycle.
- High amounts of risk and uncertainty.
- Not a good model for complex and object-oriented projects.
- Poor model for long and ongoing projects.
- Not suitable for the projects where requirements are at a moderate to high risk of changing. So, risk and uncertainty is high with this process model.
- It is difficult to measure progress within stages.
- Cannot accommodate changing requirements.
- Adjusting scope during the life cycle can end a project.
- Integration is done as a "big-bang" at the very end, which doesn't allow identifying any technological or business bottleneck or challenges early.

□ **When to Use**

- Requirements are clearly defined and fixed.
- Technology stack is well understood.
- Regulatory or compliance-heavy projects.
- Low customer involvement during development.

□ **Real-World Examples**

- **Government systems** (tax systems, defense applications)
- **Banking core systems**
- **Embedded systems** with fixed functionality
- **Payroll systems**

□ **Why It Works Here**

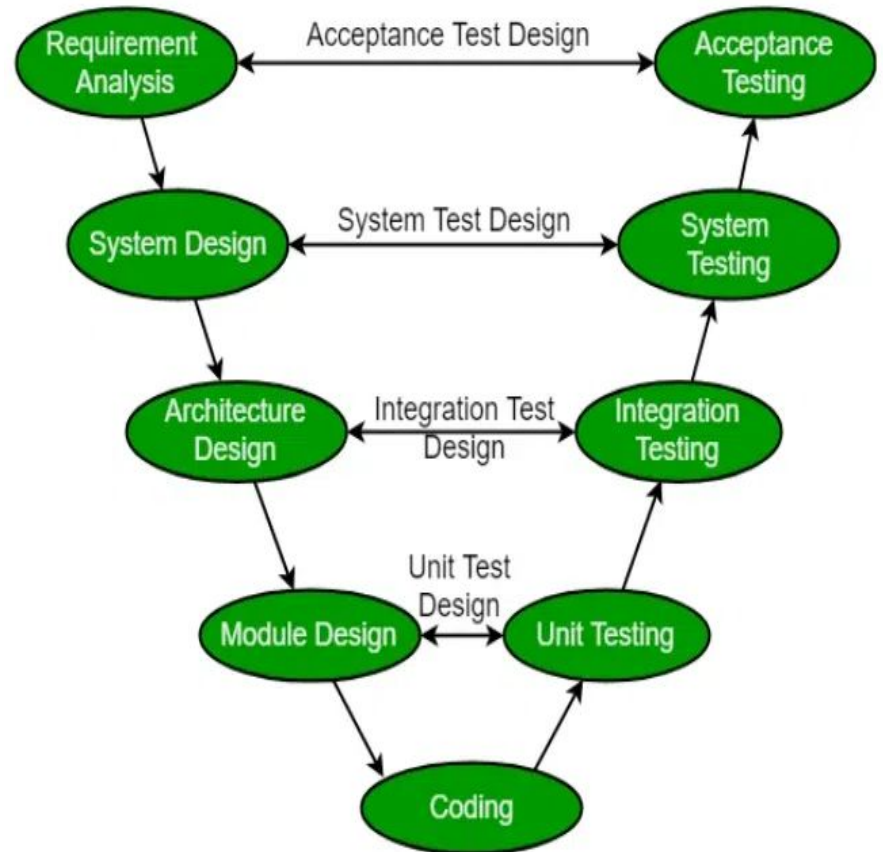
- Clear documentation is mandatory.
- Predictable timelines and budgets.
- Easy to manage and audit.

□ **Limitations**

- Difficult to accommodate requirement changes.
- Testing occurs late, increasing risk.

SDLC V-Model

- It is a method that includes testing and validation alongside each development phase. It creates a structure like the letter 'V,' which includes various phases that we will discuss in detail.
- **Phases of SDLC V-Model**
- The V-Model, which includes the Verification and Validation it is a structural approach to software development. The following are the different Phases of the V-Model of the SDLC.



Core Phases of the V-Model

- **Left Side (Verification):** Focuses on requirements gathering, system design, and detailed technical specifications.
- **Intersection (Coding):** The actual implementation of the software modules based on the design.
- **Right Side (Validation):** Executes testing plans created during the verification stages, including Unit, Integration, System, and Acceptance Testing.

Advantages of V-Model

- V-Model is used for small projects where project requirements are clear.
- Simple and easy to understand and use.
- This model focuses on verification and validation activities early in the life cycle thereby enhancing the probability of building an error-free and good quality product.
- It enables project management to track progress accurately.

Disadvantages of the V-Model

- The V-Model is a linear and sequential model, which can make it challenging to adapt to changing requirements or unforeseen events.
- The V-Model can be time-consuming, as it requires a lot of documentation and testing.
- High risk and uncertainty.
- It is not good for complex and object-oriented projects.
- It is not suitable for projects where requirements are not clear and contain a high risk of changing.
- This model does not support iteration of phases.
- The V-Model places a strong emphasis on documentation, which can lead to an overreliance on documentation at the expense of actual development work.

□ **When to Use**

- Strong emphasis on testing and validation.
- Requirements are well defined.
- Safety-critical applications.

□ **Real-World Examples**

- **Medical devices** (MRI software, patient monitoring)
- **Automotive software** (ABS, airbag systems)
- **Aviation systems**

□ **Why It Works Here**

- Early test planning reduces defects.
- Ensures compliance with safety standards (ISO, FDA, DO-178C).

□ **Limitations**

- Rigid structure.
- Costly changes once development starts.

Incremental Model

- Incremental Model is a process of software development where requirements divided into multiple standalone modules of the software development cycle. In this model, each module goes through the requirements, design, implementation and testing phases. Every subsequent release of the module adds function to the previous release. The process continues until the complete system achieved.

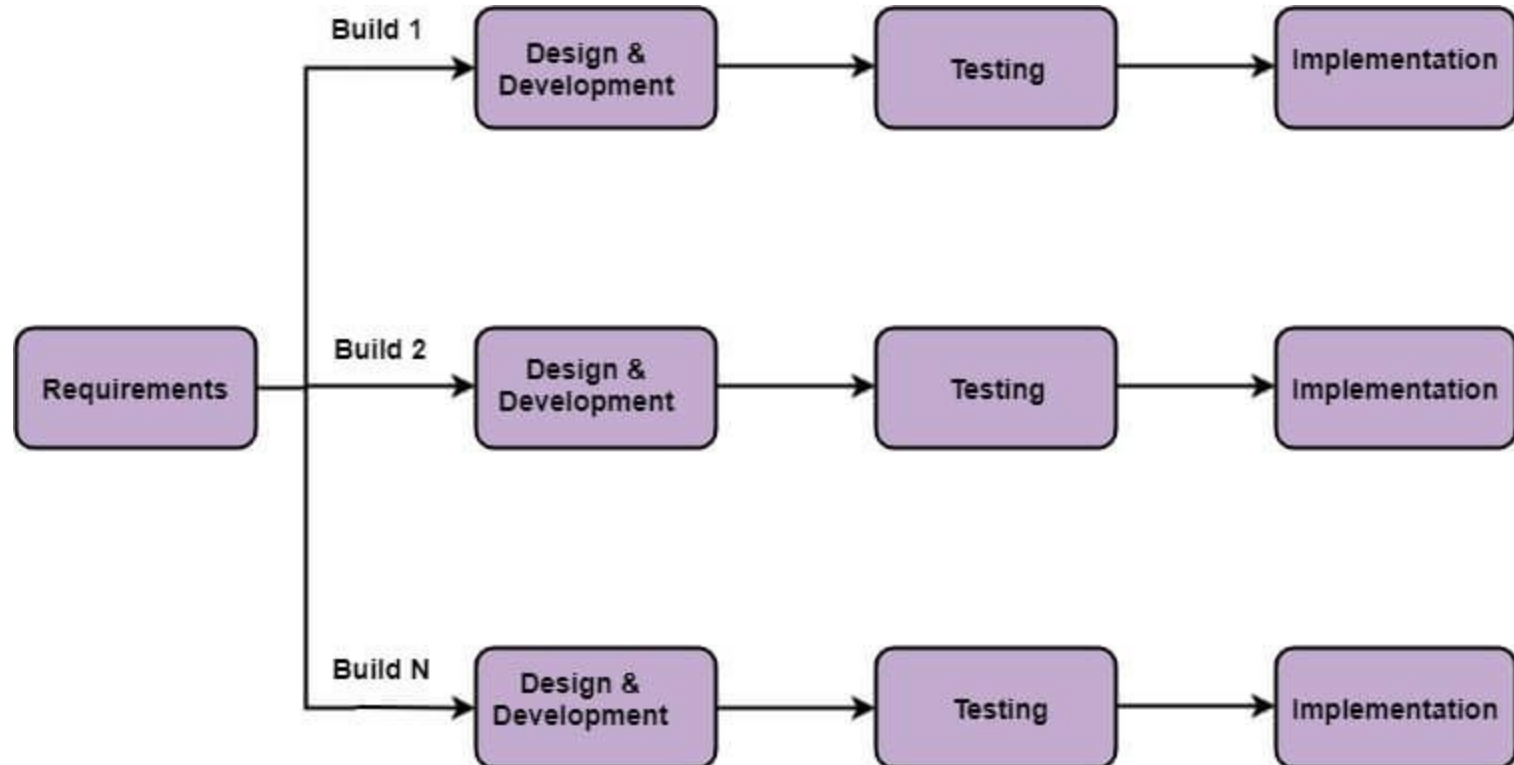


Fig: Incremental Model

- The various phases of incremental model are as follows:
- **1. Requirement analysis:** In the first phase of the incremental model, the product analysis expertise identifies the requirements. And the system functional requirements are understood by the requirement analysis team. To develop the software under the incremental model, this phase performs a crucial role.
- **2. Design & Development:** In this phase of the Incremental model of SDLC, the design of the system functionality and the development method are finished with success. When software develops new practicality, the incremental model uses style and development phase.
- **3. Testing:** In the incremental model, the testing phase checks the performance of each existing function as well as additional functionality. In the testing phase, the various methods are used to test the behavior of each task.
- **4. Implementation:** Implementation phase enables the coding phase of the development system. It involves the final coding that design in the designing and development phase and tests the functionality in the testing phase. After completion of this phase, the number of the product working is enhanced and upgraded up to the final system product

Advantage of Incremental Model

Errors are easy to be recognized.

Easier to test and debug

More flexible.

Simple to manage risk because it handled during its iteration.

The Client gets important functionality early.

Disadvantage of Incremental Model

Need for good planning

Total Cost is high.

Well defined module interfaces are needed.

□ **When to Use**

- Early delivery of working software is required.
- Budget or time constraints exist.
- User feedback is important but not continuous.

□ **Real-World Examples**

- **E-commerce platforms**
- **Customer relationship management (CRM) systems**
- **Enterprise dashboards**

□ **Why It Works Here**

- Reduces initial delivery time.
- Allows partial system use early.
- Easier risk management.

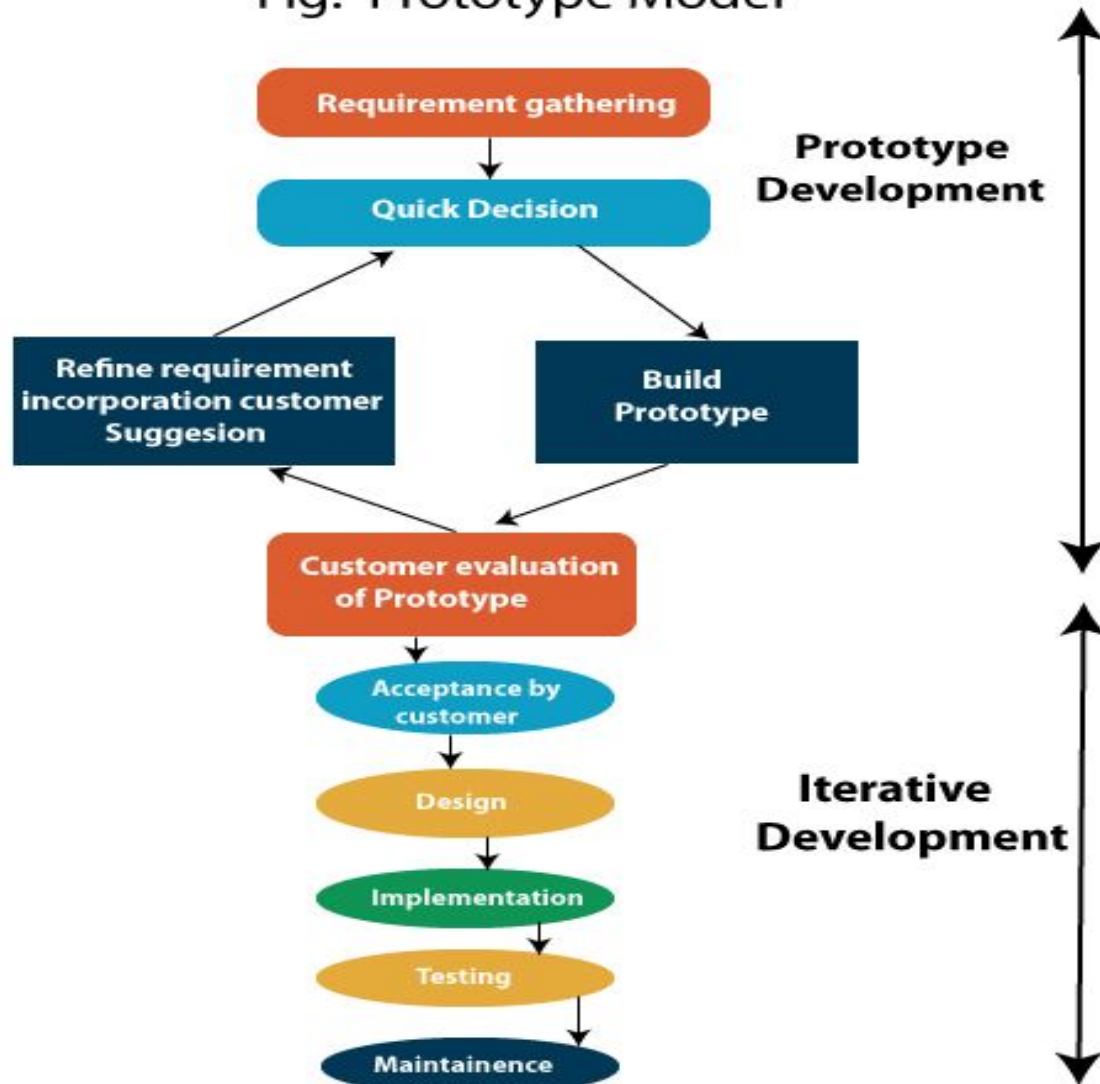
□ **Limitations**

- Requires good architectural planning.
- Integration can become complex.

Prototype model

- The prototype model requires that before carrying out the development of actual software, a working prototype of the system should be built. A prototype is a toy implementation of the system. A prototype usually turns out to be a very crude version of the actual system, possibly exhibiting limited functional capabilities, low reliability, and inefficient performance as compared to actual software. In many instances, the client only has a general view of what is expected from the software product. In such a scenario where there is an absence of detailed information regarding the input to the system, the processing needs, and the output requirement, the prototyping model may be employed.

Fig: Prototype Model



Steps of Prototype Model

- Requirement Gathering and Analyst
- Quick Decision
- Build a Prototype
- Assessment or User Evaluation
- Prototype Refinement
- Engineer Product

Advantage of Prototype Model

- Reduce the risk of incorrect user requirement
- Good where requirement are changing/uncommitted
- Regular visible process aids management
- Support early product marketing
- Reduce Maintenance cost.
- Errors can be detected much earlier as the system is made side by side.

Disadvantage of Prototype Model

- An unstable/badly implemented prototype often becomes the final product.
- Require extensive customer collaboration
 - Costs customer money
 - Needs committed customer
 - Difficult to finish if customer withdraw
 - May be too customer specific, no broad market
- Difficult to know how long the project will last.
- Easy to fall back into the code and fix without proper requirement analysis, design, customer evaluation, and feedback.
- Prototyping tools are expensive.
- Special tools & techniques are required to build a prototype.
- It is a time-consuming process.

spiral model

- The spiral model, initially proposed by Boehm, is an evolutionary software process model that couples the iterative feature of prototyping with the controlled and systematic aspects of the linear sequential model. It implements the potential for rapid development of new versions of the software. Using the spiral model, the software is developed in a series of incremental releases. During the early iterations, the additional release may be a paper model or prototype. During later iterations, more and more complete versions of the engineered system are produced.

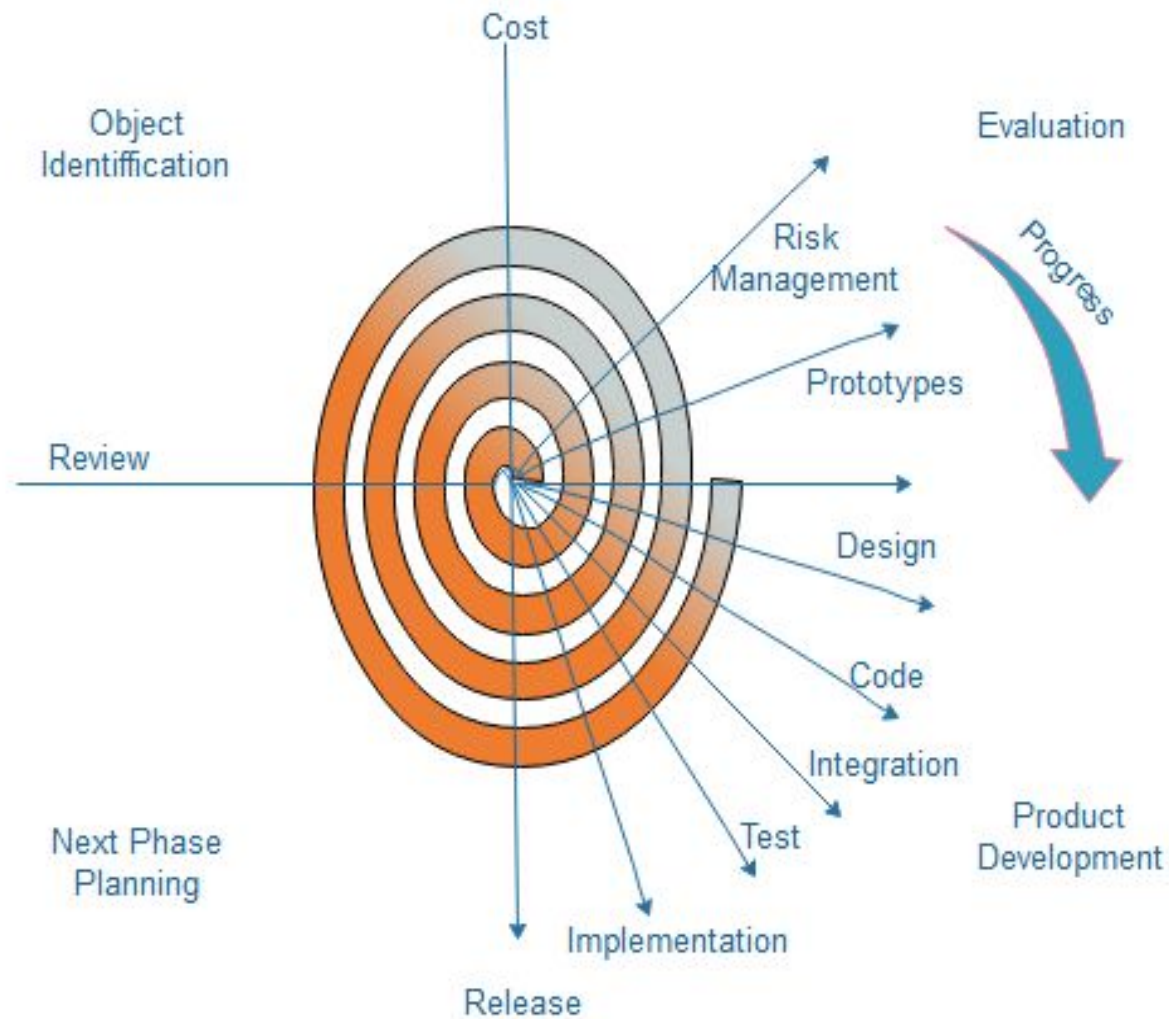


Fig. Spiral Model

How the Spiral Model Works (The Four Quadrants)

- **Planning:** Define objectives, alternatives, and constraints for the current iteration.
- **Risk Analysis:** Identify and resolve project risks (technical, business, etc.) through prototyping or analysis.
- **Engineering:** Develop and test the software increment (design, code, test).
- **Evaluation:** Review the results with the customer, decide on the next steps (commit, revise, or abandon).

Case Study Example:

E-Commerce Website Development

- **Planning:** Gather basic needs (product display, cart, checkout).
- **Risk Analysis:** Identify security/scalability risks; build a basic prototype to test data access.
- **Engineering:** Develop basic product listing, cart, and a simple payment gateway.
- **Evaluation:** Customer feedback on usability, security concerns raised.

Case Study:

- A prominent real-world application of the spiral model is the **Income Tax e-Governance System** developed by Tata Consultancy Services (TCS) for the Indian government.

- **Project Need:** The system required high security, extreme scalability to handle millions of users, and the ability to adapt to frequently changing tax laws.
- **The Approach:** TCS implemented the project in multiple cycles (spirals):
 - **First Spiral:** Focused on core modules like basic tax return filing. Small prototypes were built to test security protocols.
 - **Subsequent Spirals:** Gradually expanded the system to include complex features like payment integration and advanced audits, refining each based on government feedback and updated regulations.
- **Result:** While expensive and time-consuming, the iterative risk analysis ensured the final system was robust, secure, and compliant with complex legal requirements.

Other Notable Examples

- **Microsoft Windows:** Early versions of Windows were developed using successive iterations to incorporate customer feedback and address technical risks from previous versions.
- **NASA Space Programs:** NASA has used the spiral model for mission control software and projects like the **Space Shuttle program** and **TReK**, where operational risks are high and technical failure is unacceptable.
- **Gaming Industry:** Large-scale online games often use the spiral model to release early versions (betas) for user feedback and risk management before a full launch.

□ **When to Use**

- High technical or business risks.
- Frequent stakeholder evaluation required.
- Large-scale systems.

□ **Real-World Examples**

- **Banking platforms**
- **Telecommunication systems**
- **Defense software systems**

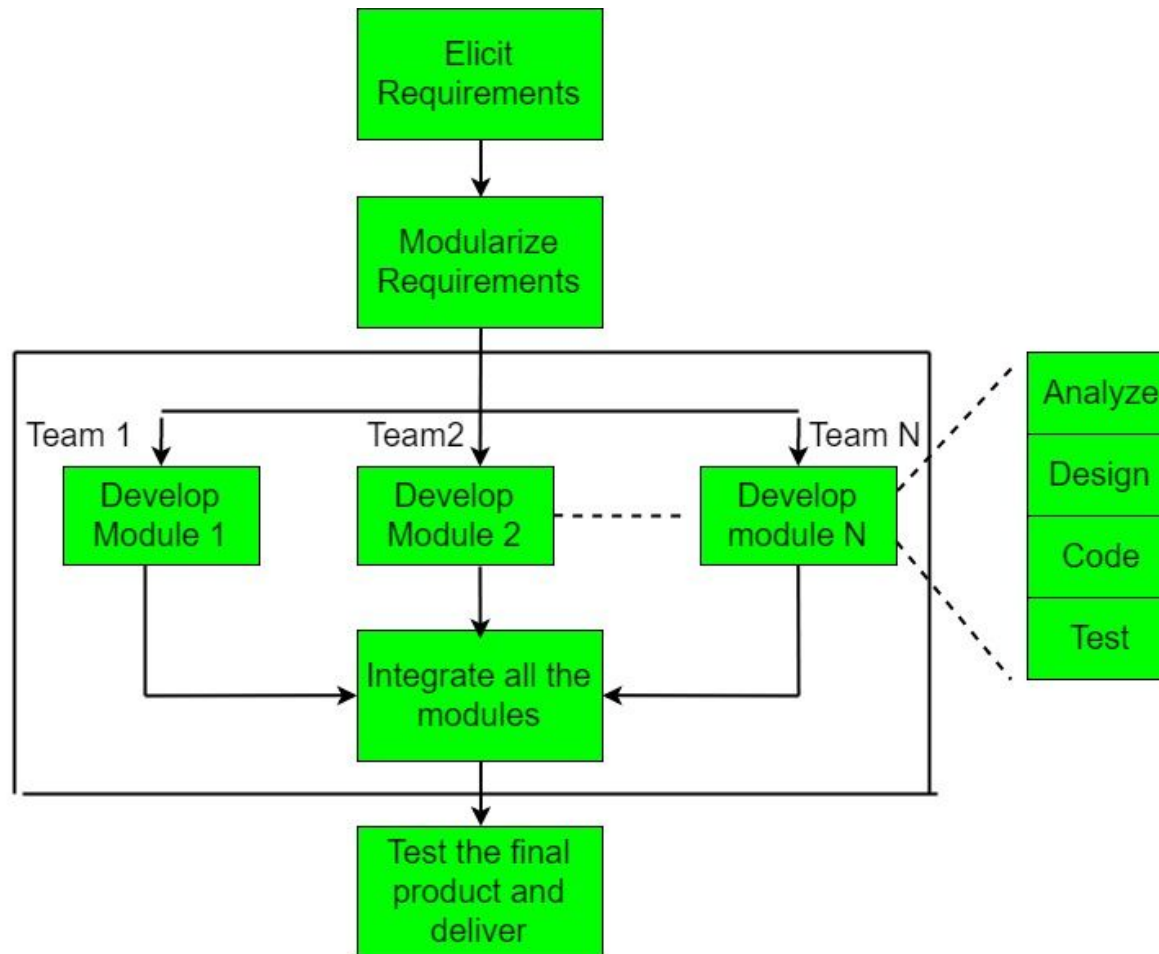
□ **Why It Works Here**

- Early identification and mitigation of risks.
- Suitable for prototyping complex features.

□ **Limitations**

- Expensive.
- Requires experienced risk analysts.

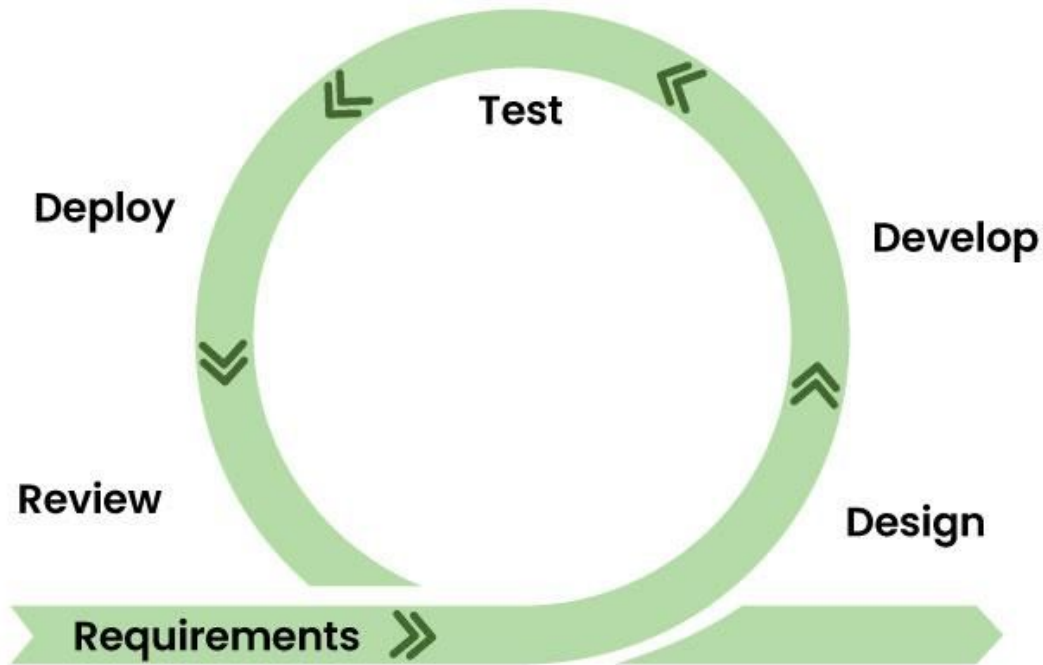
Rapid Application Development Model (RAD)



When to use the RAD Model?

- **Well-understood Requirements:** When project requirements are stable and transparent, RAD is appropriate.
- **Time-sensitive Projects:** Suitable for projects that need to be developed and delivered quickly due to tight deadlines.
- **Small to Medium-Sized Projects:** Better suited for smaller initiatives requiring a controllable number of team members.
- **High User Involvement:** Fits where ongoing input and interaction from users are essential.
- **Innovation and Creativity:** Helpful for tasks requiring creative inquiry and innovation.
- **Prototyping:** It is necessary when developing and improving prototypes is a key component of the development process.
- **Low technological Complexity:** Suitable for tasks using comparatively straightforward technological specifications.

Agile Model



Agile Model Contd..

- In the mid-1990s, the **Agile Software Development Model** was proposed to overcome these drawbacks of the **Waterfall Model**.
- **The Agile Model** was primarily designed to help a project adapt quickly to change requests. So, the main aim of the Agile model is to facilitate quick project completion.

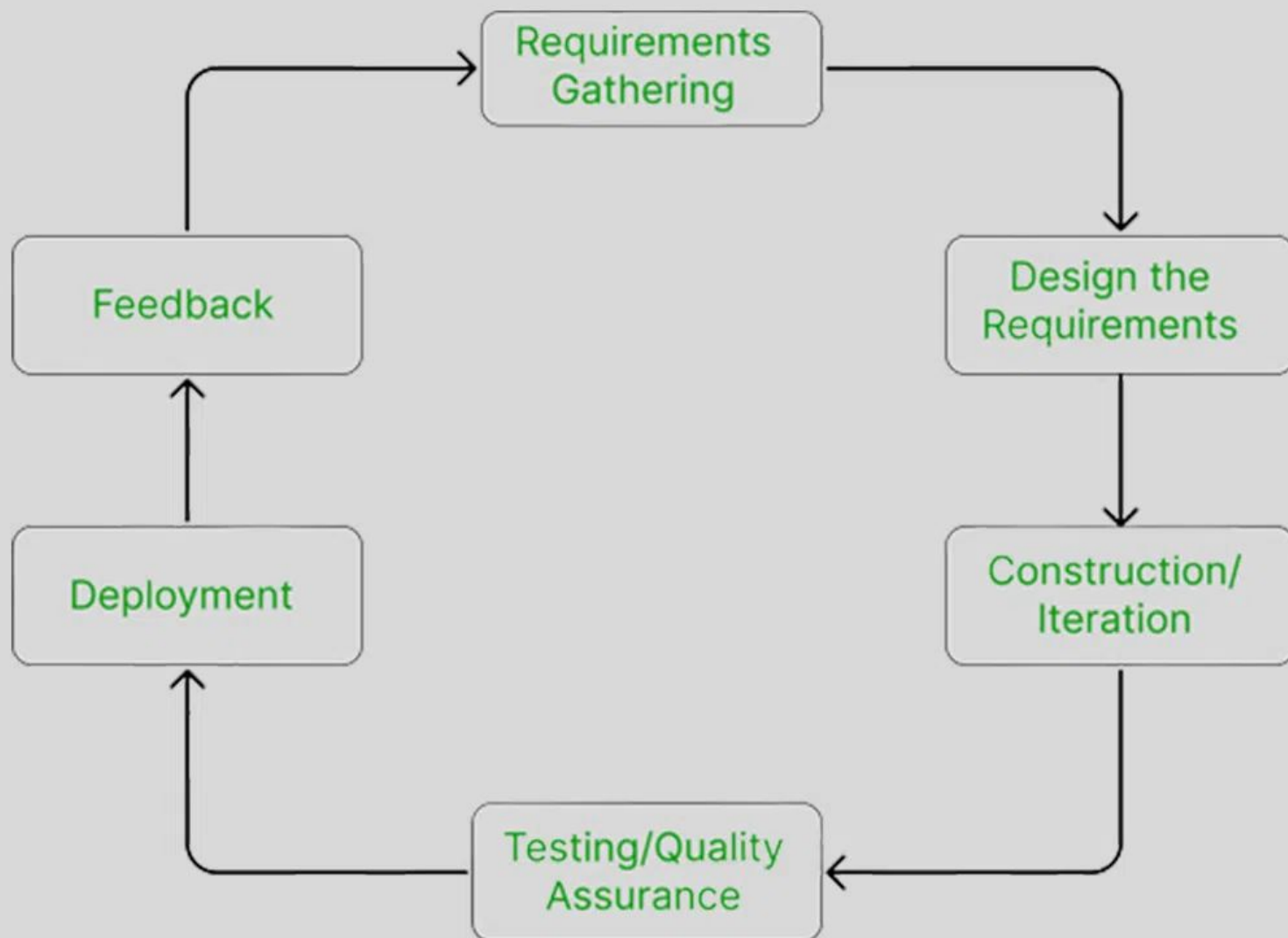


Fig:- Steps in Agile SDLC Model

Design the Requirements

- **Designing the system:** Once the requirements are gathered, the next step is to design the system's overall architecture based on those needs. This helps to verify the software is structured in a way that meets the user's expectations.
- **Creating wireframes:** Next, wireframes for the user interface (UI) are created. These are simple blueprints that show how the software will look and how users will interact with it, ensuring it's user-friendly and easy to navigate.
- **High-level design with UML diagrams:** At this stage, high-level designs using UML (Unified Modeling Language) diagrams are created to visually represent the software's structure and how different parts will work together.
- **Prototyping for feedback:** Prototypes are made to give stakeholders an early look at the software. This helps gather feedback early in the process and allows for adjustments before the full development begins.

- **Design:** The design team gives the paper sketches for the screens to be designed. Wireframes and prototypes are created once the feature list is frozen.

Case Study: Zomato

- Search the location and/or a restaurant
- List of restaurants with basic details like photo, cuisine, price
- Detailed view of restaurant with Menu, price, location, any offers, ratings, etc.
- Adding/Updating/Deleting items to and from the cart
- Select or add a new delivery address
- Review order
- Placing an order
- Payment page
- Track status of the order
- Enabling customer support chat/phone number

Construction / Iteration

- Development team members start working on their project, which aims to deploy a working product.
- Each cycle typically consist between **1-4 weeks**, and at the end, the team delivers a working version of the software.

- **Development of Features:** The team works on the features identified during the requirement and design phases.
- **Coding and Implementation:** New functionalities are coded and integrated into the software based on the goals for that specific iteration.
- **Delivering a Working Product:** After each iteration, a usable version of the software is ready.
- **Incremental Improvement:** With every cycle, the software is enhanced, adding more features and refining existing ones.

Testing / Quality Assurance

- **Unit Testing**: Unit testing is the process of checking small pieces of code to ensure that the individual parts of a program work properly on their own. Unit testing is used to test individual blocks (units) of code.
- **Integration Testing**: Integration testing is used to identify and resolve any issues that may arise when different units of the software are combined.
- **System Testing**: Goal is to ensure that the software meets the requirements of the users and that it works correctly in all possible scenarios.

Deployment

- Deploy the software to a test or live environment so that it can be used by customers or end-users.
- Make the software accessible to users, verifying they can start using it as expected.
- Verify the deployment goes smoothly and fix any issues that come up quickly

- **Deployment:** The feature is deployed and user experience is analyzed by tracking some events and taking feedback.

In **subsequent iterations**, the following filters can be added to various releases.

- Filter by place within a location
- Filter by cuisine
- Filter by restaurants ratings
- Filter by price
- Filter by popularity
- Filter by coupons applicable
- Filter by restaurant offers
- Filter by delivery times, etc.

Feedback

- **Take feedback** from customers, users, and stakeholders after each iteration.
- Understand how well the product meets user needs and identify areas for improvement.
- **Check for bugs** or issues that need fixing.
- **Make adjustments** to the development plan based on feedback to improve the product further.

Popular Agile Software Development Frameworks

- **Scrum**
- **Kanban**
- **Lean Software Development**
- **SAFe (Scaled Agile Framework)**
- **Extreme Programming (XP)**

Benefits of Agile Software Development

- **Increased flexibility:** Agile allows for adapting requirements and priorities to ensure that the final product meets customer needs.
- **Time to market:** By using iterations and incremental development valuable features or products can be delivered faster.
- **Enhanced quality:** Continuous feedback and testing together contribute to the production of higher quality deliverables.
- **Improved collaboration:** Agile promotes collaboration and communication among functional teams fostering better teamwork.
- **Heightened customer satisfaction:** Regular feedback, from customers drives product improvements and ensures customer satisfaction.

Case Studies:

1. Spotify: Crafting a Trend of Continuous Improvement
2. ING: Banking on Agile
3. LEGO: Building Success Brick by Brick
4. Microsoft: Transforming to the Cloud Era
5. Bosch: Innovating in the Industrial Sector

DevOps Model

- **Overview**

DevOps integrates **development and operations** to enable continuous delivery.

- **Use Cases**

Best for **products requiring frequent updates and high availability**.

- **When to Use**

- Continuous integration and deployment needed.
- Cloud-native applications.
- Micro services architecture.

❑ **Real-World Examples**

- Streaming platforms
- Cloud services
- Large-scale SaaS platforms

❑ **Why It Works Here**

- Faster releases.
- Improved system reliability.
- Automated testing and deployment.

❑ **Limitations**

- Cultural shift required.
- Tooling complexity.

Summary Table

SDLC Model	Best Use Case
Waterfall	Fixed requirements, regulated domains
V-Model	Safety-critical systems
Incremental	Feature-based delivery
Iterative	Evolving requirements
Spiral	High-risk, complex systems
Agile	Dynamic, customer-driven products
RAD	Rapid prototyping
DevOps	Continuous delivery systems

What is software project management?

- Software project management is an art and discipline of planning and supervising software projects. It is a sub-discipline of software project management in which software projects planned, implemented, monitored and controlled.
- It is a procedure of managing, allocating and timing resources to develop computer software that fulfills requirements.

Prerequisite of software project management?

There are three needs for software project management. These are:

- Time
- Cost
- Quality

It is an essential part of the software organization to deliver a quality product, keeping the cost within the client's budget and deliver the project as per schedule. There are various factors, both external and internal, which may impact this triple factor. Any of three-factor can severely affect the other two.

Project Manager

A project manager is a character who has the overall responsibility for the planning, design, execution, monitoring, controlling and closure of a project. A project manager represents an essential role in the achievement of the projects.

A project manager is a character who is responsible for giving decisions, both large and small projects. The project manager is used to manage the risk and minimize uncertainty. Every decision the project manager makes must directly profit their project.

Role of a Project Manager:

1. Leader

A project manager must lead his team and should provide them direction to make them understand what is expected from all of them.

2. Medium: The Project manager is a medium between his clients and his team. He must coordinate and transfer all the appropriate information from the clients to his team and report to the senior management.

3. Mentor:

He should be there to guide his team at each step and make sure that the team has an attachment. He provides a recommendation to his team and points them in the right direction.

Responsibilities of a Project Manager:

Managing risks and issues.

Create the project team and assigns tasks to several team members.

Activity planning and sequencing.

Monitoring and reporting progress.

Modifies the project plan to deal with the situation.

- Activities
- Software Project Management consists of many activities, that includes planning of the project, deciding the scope of product, estimation of cost in different terms, scheduling of tasks, etc.
- **The list of activities are as follows:**
 - Project planning and Tracking
 - Project Resource Management
 - Scope Management
 - Estimation Management
 - Project Risk Management
 - Scheduling Management
 - Project Communication Management
 - Configuration Management